

Uffizi RL: The Agents in Full

States, Action Spaces, Policies, Reward Functions, the Maths, and the Code

Marco Canal | Reinforcement Learning, BSE

Spring 2026

This is the single place that answers “what is each agent, what does it observe, what can it do, and what is it optimising.” Every reward weight and line reference is taken from the current code. Code listings are *lightly stylized* (renamed for clarity, error handling stripped); the file and line in each heading points to the real source. The companion map of where everything lives is `docs/AGENTS.md`.

1 The cast, at a glance

The project has two algorithm families and one layered environment design: a single core environment (`UffiziEnv`) holds the real state and reward, and three thin wrappers change only the action interface, delegating the per-minute reward back to the core. The toy world (`ToyTabularEnv`) is separate, small enough for an exact table.

Agent	Env class	State	Action	Algorithm	γ
Toy art lover	<code>ToyTabularEnv</code>	5-tuple ($\sim 123k$)	<code>Discrete(1+deg)</code>	Q-learning, Double-Q	0.99
Deep navigator	<code>UffiziEnv</code>	$\mathbb{R}^{5N+3}$	<code>Discrete(1+deg)</code>	MaskablePPO, PPO, DQN	0.995
Art-lover booking	<code>RamaArtLoverEnv</code>	$\mathbb{R}^{25}$	<code>Discrete(4)</code>	MaskablePPO	0.999
Tourist booking	<code>RamaTouristEnv</code>	$\mathbb{R}^{25}$ (inherit.)	<code>Discrete(4)</code>	MaskablePPO	0.999
Baseline walk	<code>PlannedRouteEnv</code>	$\mathbb{R}^8$	<code>Discrete(2)</code>	PPO/DQN (no levers)	0.995/.999

Two families. *Value-based* methods learn an action-value $Q(s, a)$ and act greedily (tabular Q-learning $\rightarrow$ Double Q-learning $\rightarrow$ DQN). *Policy-based* actor-critic methods learn the policy $\pi_\theta(a | s)$ directly (PPO $\rightarrow$ MaskablePPO). We use the tabular pair for the foundations proof, and the deep pair for the real museum, with MaskablePPO as the headline and DQN / unmasked PPO as controls.

2 Part I. Foundations: the toy museum (`agents/q_learning.py`)

A 12-room graph with deterministic Gaussian crowd waves. Small enough that Q is a literal lookup table, so we can prove the core idea (*temporal arbitrage*: time your masterpiece visit to its crowd valley) before scaling.

2.1 The MDP

State (`get_state`, `q_learning.py:304`). A discrete 5-tuple, small enough that the whole value function fits in a table of $\approx 1.23 \times 10^5$ cells:

$$s = \left(\underbrace{\text{room}}_{12}, \underbrace{t_{\text{bin}}}_{0..9}, \underbrace{c_{\text{bott}}}_{0..3}, \underbrace{c_{\text{leo}}}_{0..3}, \underbrace{\text{visited}}_{0..63} \right).$$

Reading each field, and why the agent needs it:

- **room** (1 of 12): where I am standing now, my only sense of location.

- t_{bin} (0–9): how far through the day, in ten coarse slices. The crowd waves peak at fixed hours, so the agent must know the time to exploit a valley.
- c_{bott} (0–3): how crowded the Botticelli pair is *right now*, bucketed into empty, moderate, busy, or packed. This is what makes “wait for the peak to pass” decidable.
- c_{leo} (0–3): the same live crowd reading for Leonardo.
- **visited** (0–63): a 6-bit flag of which of the 6 must-sees I have already seen, so the agent stops revisiting and knows what is still left.

Crowd is coarsened to 4 levels and time to 10 slices on purpose: it keeps the table small enough to fill by experience.

Actions (q_learning.py:155). $a \in \{0, \dots, \text{max_degree}\}$: 0 is *stay*, 1... move to the i -th alphabetically-sorted neighbour. A legality mask $\mathcal{A}(s)$ marks valid moves.

Reward / objective (q_learning.py:419). The agent maximises the sum over the day of a per-minute utility with four parts:

$$r(s, a) = \underbrace{\frac{\text{imp}(\text{room}) \cdot \nu}{1 + \alpha_A \rho^2}}_{\text{art value}} \underbrace{- c_{\text{step}}}_{\text{urgency}} \underbrace{- \kappa_{\text{close}} \max(0, d_{\text{exit}} + b - \tau)}_{\text{egress pressure}} + \underbrace{r_{\text{exit}} \text{ OR } r_{\text{noexit}}}_{\text{terminal}}.$$

- **art value:** the room’s importance to this visitor, divided down when crowded (ρ is the live density, $\alpha_A = 6$ makes the art lover strongly crowd-averse). The satiation factor $\nu = (\text{novelty_decay})^{n_{\text{visits}}}$ halves on every revisit, so re-entering a seen room pays almost nothing: the agent is pushed to cover new rooms, not loop.
- **urgency** ($-c_{\text{step}}$ each minute): standing still or backtracking is a small net loss, so the visit keeps moving.
- **egress pressure:** zero while there is still time to walk to a door (d_{exit} hops plus a buffer b , within the time left τ), and growing once the agent is too far out as closing nears.
- **terminal:** +15 for leaving voluntarily, -50 for being trapped inside at closing.

Constants (q_learning.py:129): $\alpha_A = 6$, $\text{novelty_decay} = 0.5$, $c_{\text{step}} = 0.5$, $\kappa_{\text{close}} = 4$, $b = 5$, $r_{\text{exit}} = +15$, $r_{\text{noexit}} = -50$.

2.2 Q-learning (q_learning.py:520)

Model-free, off-policy, value-based. We bootstrap each cell toward the Bellman target:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a' \in \mathcal{A}(s')} Q(s', a')}_{\text{TD target}} - Q(s, a) \right]$$

with $\alpha = 0.1$, $\gamma = 0.99$, and ε -greedy exploration over valid actions ($\varepsilon : 1.0 \rightarrow 0.01$, multiplied by 0.9995 each episode). The greedy evaluation policy is $\pi(s) = \arg \max_{a \in \mathcal{A}(s)} Q(s, a)$. Note the max is *masked* to legal moves, so we never bootstrap off an impossible transition.

```

1 def get_state(self):
2     time_bin = min(9, int(10 * self.t / self.horizon)) # time of day, 0..9
3     crowd_bott = bin(max(crowd("A11"), crowd("A12"))) # Botticelli pair, 0..3
4     crowd_leo = bin(crowd("A35")) # Leonardo, 0..3
5     visited = bitmask(self.visited, KEY_ROOMS) # 6 must-sees -> 0..63
6     return (self.current_room, time_bin, crowd_bott, crowd_leo, visited)
7
8 # --- the update (epsilon-greedy action, then masked TD(0)) ---
9 td_target = r if done else r + gamma * max(Q[s_next][valid]) # masked max
10 Q[s][a] += lr * (td_target - Q[s][a]) # lr = 0.1

```

2.3 Double Q-learning (q_learning.py:826)

The single max over noisy estimates is biased upward: the same noisy table both *selects* and *evaluates* the next action, and by Jensen’s inequality $\mathbb{E}[\max_a Q(s', a)] \geq \max_a \mathbb{E}[Q(s', a)]$. Double Q-learning keeps two tables and decouples the two roles:

$$Q_A(s, a) \leftarrow Q_A(s, a) + \alpha \left[r + \underbrace{\gamma Q_B(s', \arg \max_{a'} Q_A(s', a'))}_{\text{select with } A} - Q_A(s, a) \right],$$

(Q_B evaluates what Q_A selects; roles swapped on a coin flip). The policy acts on the average $\frac{1}{2}(Q_A + Q_B)$.

```

1 if coin() < 0.5:
2     a_star = argmax(Q_A[s_next][valid])      # SELECT with table A
3     target = r + gamma * Q_B[s_next][a_star] # EVALUATE with table B
4     Q_A[s][a] += lr * (target - Q_A[s][a])
5 else:
6     a_star = argmax(Q_B[s_next][valid])      # symmetric: roles swapped
7     target = r + gamma * Q_A[s_next][a_star]
8     Q_B[s][a] += lr * (target - Q_B[s][a])

```

Honest result. On this *deterministic* toy there is little maximisation bias to remove, so vanilla Q-learning slightly edges Double-Q (100.5 vs 86.9). Double-Q earns its keep under noise, which is exactly where its deep descendant (Double DQN) helps. It is the tabular ancestor of that idea.

3 Part II. The core environment (environment/uffizi_env.py)

The full 98-room museum. One controlled visitor; the rest of the crowd is the (off-limits) simulator advancing one minute per step. The toy table cannot survive 98 rooms with a continuous density each, so Q/π become neural networks.

3.1 State (_build_observation, uffizi_env.py:573)

The observation is one float vector $o(s) \in \mathbb{R}^{5N+3}$ ($N = 98$ rooms), the policy’s entire view of the museum each minute, eight blocks stacked together:

$$o(s) = [\text{onehot}(\text{room}); \rho; \Delta\rho; t/T; \text{slack}; \text{visited}; \text{progress}; \text{fatigue}].$$

- **onehot(room)** (N dims, $\{0, 1\}$): which room I occupy. Everything else is read relative to this.
- ρ , **density** (N , $[0, 1]$): how full *every* room is now (occupancy over capacity), so the agent can steer toward calmer rooms.
- $\Delta\rho$, **trend** (N , $[-1, 1]$): whether each room is filling or emptying, so it can anticipate a room about to clear rather than react late.
- t/T , **time** (1 , $[0, 1]$): how far through the open day, the only hard deadline.
- **slack, egress** (1 , $[-1, 1]$): minutes left minus hops to the nearest exit. Positive means “still time to sightsee”, ≤ 0 means “head for a door now”. This one feature is what lets the policy time its exit.
- **visited** (N , $\{0, 1\}$): which rooms I have already entered.
- **progress, appreciation** (N , $[0, 1]$): how much of each room’s ideal dwell I have already used up. Without this, “stay until satisfied, then move on” is not decidable from the state, this block is the fix for that observability bug.
- **fatigue** (1 , $[0, 1]$): accumulated tiredness over the visit.

Actions (uffizi_env.py:326). `Discrete(1+max_degree)`: stay, or move to a neighbouring room. A legality mask gives the set $\mathcal{A}(s)$ of one-hop moves.

3.2 Reward / objective (compute_reward, uffizi_env.py:833)

The per-minute reward is a sum of interpretable terms,

$$r = r_{\text{art}} + r_{\text{time}} + r_{\text{cong}} + r_{\text{close}} + r_{\text{complete}} + r_{\text{bored}} + r_{\text{checkin}} + r_{\text{wait}}$$

The headline term is the art value, crowd-discounted and satiating:

$$r_{\text{art}} = \text{imp} \cdot \underbrace{\frac{1}{1 + \alpha \rho^2}}_{\text{crowd factor (gentle, } \alpha=0.5)} \cdot \underbrace{\text{clip}\left(\frac{D_{\text{ideal}} - e}{\text{shoulder}}, 0, 1\right)}_{\text{novelty: plateau satiation}}, \quad D_{\text{ideal}} = \max(1, k_d(\text{imp} - \text{floor})).$$

Appreciation accumulates *slower in a crowd* (fight-to-the-front), so a packed room eats more of the fixed day: $e \leftarrow e + \frac{1}{1+\beta\rho}$ (a busier day therefore yields fewer rooms and fewer total points). Reading every term:

- **r_{art} , the payoff:** how much this visitor values the room (importance), times two factors. The *crowd factor* $1/(1 + \alpha\rho^2)$ shaves value when the room is full, but *gently* ($\alpha = 0.5$), so a packed Botticelli still far outscores an empty side gallery (a harsh penalty made the agent skip masterpieces). The *novelty* factor pays the room’s full value across its ideal dwell D_{ideal} (about 30 minutes for a masterpiece, scaling with importance) and tapers to zero only over the last few minutes, so “stay until satisfied, then leave” beats a quick glance.
- **r_{time} , urgency** (-0.15 per minute): a flat per-minute cost, so lingering past satiation, or wandering, is a net loss.
- **r_{close} , egress** ($-\kappa \max(0, d_{\text{exit}} + b - \tau)$, $\kappa=4$): zero while a door is comfortably reachable, escalating once the agent falls behind schedule, the learnable “head out in time” signal.
- **r_{complete} , finish a must-see** ($+k_c \text{imp}$, $k_c=15$, only for $\text{imp} \geq 7$): a one-time bonus for fully appreciating a masterpiece. It is the discrete target that makes the agent see one to the end rather than glance, then move to the next.
- **r_{bored} , stop camping** ($-k_b$ if it chooses to stay in an already-satiated room): with r_{complete} pulling it to stay until done and boredom pushing it to leave after, the dwell sharpens to about D_{ideal} and “camp one room forever” is killed. Off in the bare core, on ($k_b=4$) for the booking agents.
- **r_{checkin} , r_{wait} , booking** (Part III): $+50$ for entering a booked masterpiece inside its window, and -2 for idling before that window opens. Zero unless RAMA is active.
- **$r_{\text{cong}} = 0$:** there is deliberately no separate congestion term. Crowd already enters through r_{art} and the slower dwell above; adding one double-counted aversion and made the agent skip crowded masterpieces.

```

1 crowd_factor = 1.0 / (1.0 + art_crowd_alpha * dens**2)           # gentle: alpha = 0.5
2 ideal_dwell  = max(1.0, dwell_per_importance * max(0.0, importance - floor))
3 novelty      = clip((ideal_dwell - extracted) / shoulder, 0.0, 1.0) # plateau
4 extracted    += 1.0 / (1.0 + dwell_crowd_beta * dens)           # crowded -> slower
5 r_art        = importance * crowd_factor * novelty
6 r_close      = -closing_pressure * max(0.0, dist_exit + buffer - time_left)
7 reward       = r_art + r_time + r_close + r_completion + r_boredom + r_checkin + r_wait

```

3.3 MaskablePPO: the headline algorithm (agents/train_maskable_ppo.py)

PPO is an on-policy actor-critic: an *actor* $\pi_\theta(a \mid s)$ proposes moves, a *critic* $V_\theta(s)$ scores states. The policy-gradient direction is

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) \hat{A}(s, a)],$$

with advantages estimated by GAE,

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad \hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}.$$

PPO maximises a *clipped surrogate* that forbids destructively large policy steps (its trust region):

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad \rho_t = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

Action masking (the decisive trick). The museum is a graph, so $\approx 80\%$ of the ~ 100 action indices are illegal at any state. We set the illegal logits to $-\infty$ *before* the softmax,

$$\pi_\theta(a | s) = \frac{e^{z_a} \mathbb{1}[a \in \mathcal{A}(s)]}{\sum_{a' \in \mathcal{A}(s)} e^{z_{a'}}},$$

so the policy never spends probability mass on impossible moves. This is exactly the same idea as the masked max in the toy Q-update, lifted into the softmax.

```

1 logits[~legal_actions(state)] = -inf      # zero out illegal moves...
2 pi = softmax(logits)                     # ...so pi has support only on A(s)
3
4 MaskablePPO("MlpPolicy", env,
5     learning_rate = lambda p: 1e-4 + 1e-4 * p, # linear 2e-4 -> 1e-4
6     n_steps = 4096, batch_size = 512, n_epochs = 8,
7     gamma = 0.995, gae_lambda = 0.98, clip_range = 0.2,
8     ent_coef = 0.005, policy_kwargs = {"net_arch": [256, 256]})

```

Curriculum (train_maskable_ppo.py:227). The crowd is ramped over three stages, $0.36 \rightarrow 0.64 \rightarrow 1.0$ of peak (15%/35%/50% of the step budget); episodes are always the full 615-minute day, and training uses random start rooms (exploring starts). **Evaluation** uses common random numbers (CRN): every model is scored on the identical fixed crowds (seeds $900000+i$), so reward gaps reflect the policy, not luck.

3.4 The controls: unmasked PPO and DQN (agents/train_ablations.py)

To isolate the value of masking and of the algorithm family, two controls share the environment, reward, budget, and CRN evaluation.

Unmasked PPO. Identical PPO hyperparameters and curriculum, but *no* mask: it may select any of the ~ 100 actions and pays `invalid_action_penalty = -0.4` for illegal ones, which it must learn to avoid (wasting gradient).

DQN (value-based, off-policy). Replaces the table with $Q(s, a; \theta)$ and minimises the squared Bellman error against a *frozen target network* θ^- , sampling decorrelated transitions from a replay buffer $\mathcal{D}$:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right].$$

```

1 DQN("MlpPolicy", env, learning_rate = 2.5e-4,
2     buffer_size = 200_000, batch_size = 128, learning_starts = 5_000,
3     gamma = 0.995, policy_kwargs = {"net_arch": [256, 256]})
4 # replay buffer -> breaks temporal correlation between consecutive minutes
5 # target network -> a frozen Q for the bootstrapped max (a stationary target)

```

The two innovations PPO does *not* need: it is on-policy (fresh rollouts, no replay) and has no bootstrapped max in the actor loss (no target network); the clip handles its only instability, the policy step size. The overestimation that the target max can cause is exactly why DQN is the crowd-fragile, high-variance competitor in our results, and why Double-Q (above) exists.

4 Part III. The RAMA booking agents (the centerpiece)

RamaArtLoverEnv (environment/rama_art_lover_env.py) wraps the core env. The visitor already knows the map; the genuine decision is the *reservation*: how far ahead to book, which masterpieces, and how to pace the visit to hit each booked window. RamaTouristEnv is the same env with the tourist’s taste swapped in.

4.1 A two-phase MDP

Phase 1, booking. One lead-time choice then a book/decline choice per masterpiece; no museum time passes. **Phase 2, the visit:** walk the museum (stay / move-on) and check in inside each booked window.

State (`_obs`, rama_art_lover_env.py:278). $o \in \mathbb{R}^{25}$, in four readable groups:

- **Where I am in the decision** (3 dims): the phase flag (booking vs walking), which masterpiece I am currently deciding on, and the lead time chosen so far.
- **The booking screen** (4 dims): one availability number per lead option (1 / 7 / 21 / 35 days), each saying whether a slot is still open at that lead *given today’s crowd*. Reading this is how the agent learns “book earlier when busier”.
- **What I have locked in** (6 dims): the three window start-times I hold, and three flags for whether I have checked in to each.
- **My situation right now** (12 dims): museum time; the access read of the room I am standing in (is it a masterpiece, is it unbooked, can I enter now, minutes until its window opens); my three frozen arrival-time estimates (when I expect to reach each masterpiece); and the local readout of the current room (how drained it is, its importance, its crowd, and egress slack).

Actions (rama_art_lover_env.py:78). Discrete(4), reinterpreted by phase:

$$a \mapsto \begin{cases} \text{lead} = \text{LEAD_DAYS}[a] \in \{1, 7, 21, 35\} \text{ days} & \text{booking step 0,} \\ \text{book } (a=0) \text{ or decline } (a=1) \text{ of masterpiece } g & \text{booking steps 1, 2, 3,} \\ \text{stay } (a=0) \text{ or move-on } (a=1) & \text{walk phase.} \end{cases}$$

The fill curve. A slot is bookable only if the lead time clears a crowd-dependent threshold, so a busier day forces a longer lead:

$$\text{available}(g, \text{lead}) = \mathbb{I}\left[\text{lead} \geq \text{lead}_{\max} \cdot p(g) \cdot \frac{\text{daily_total}}{2500}\right].$$

4.2 Reward / objective

The total has two parts. **During the walk**, the agent simply collects the *core* reward of Part II, so a booked masterpiece pays its full ~ 30 -minute value the moment it checks in (the r_{checkin} bonus plus the usual r_{art}). **At booking**, three extra terms are added (rama_art_lover_env.py:355):

$$r_{\text{book}} = -d_{\text{decline}} n_{\text{decline}} - \underbrace{k_{\text{mm}} \sum_{g \in \mathcal{B}} |w_g - \hat{t}_g|}_{\text{immediate off-pace penalty}} - d_{\text{noshow}} n_{\text{noshow}}$$

- **decline** ($-d_{\text{decline}}$ per masterpiece skipped): a fixed regret for not booking a famous room (active only for a subtype that is allowed to decline).
- **off-pace mismatch** (the decisive term): for each booked masterpiece g , the gap between the slot it actually got, w_g , and the time the agent expects to be there, $\hat{t}_g$, summed and charged *at the moment of committing*. In words: if you book a slot far from when you will really arrive,

you feel it immediately. This is what turns the otherwise days-delayed “book early” lesson into a same-step gradient the optimiser can follow.

- **no-show** ($-d_{\text{noshow}}$ per masterpiece booked but never entered): reserving a slot then failing to reach it in time is penalised.

Here $\mathcal{B}$ is the set of booked masterpieces, w_g the resolved window start, and $\hat{t}_g$ the agent’s frozen arrival estimate for g . Weights: $k_{\text{mm}} = 2$, $d_{\text{decline}} = 200$ (art) / 50 (tourist), $d_{\text{noshow}} = 100$ (art) / 80 (tourist); discount $\gamma = 0.999$ (the payoff lands far downstream, at check-in).

```

1 if phase == "booking":
2     if step == 0:
3         lead = LEAD_DAYS[a]                # {1, 7, 21, 35} days
4     else:
5         g = GROUPS[step - 1]                # bott, leo, raph
6         book[g] = not (a == 1 and allow_decline) # decline only if a subtype allows it
7         if last_masterpiece:
8             commit()                        # resolve + space the windows
9             mismatch = sum(abs(window[g] - est[g]) for g in booked)
10            step_r = -decline_penalty * n_declined - mismatch_k * mismatch
11 else: # walk phase: delegate to the core env
12     inner_action = MOVE_ON if a == 1 else STAY
13     _, reward, done, _, _ = self.inner.step(inner_action) # core reward of Part II
14
15 # crowd-dependent availability seen on the booking screen:
16 def available(slot, lead):
17     return lead >= LEAD_MAX * popularity(slot) * (daily_total / 2500)

```

4.3 Policy as a type, not coercion

For both profiles `allow_decline = False`: a free decline button is an RL do-nothing trap (the optimiser collapses to declining all three, since skipping is immediately safe and the payoff is delayed). So “always books the three masterpieces” is a *type characterisation* (nobody flies to Florence to skip Botticelli, Leonardo and Raphael); the genuine learned decision is the *lead time*. The arrival prior $\hat{t}_g$ is computed once from reference walks and *frozen* (`_arr_alpha = 0`) to avoid a self-referential runaway.

4.4 Art lover vs tourist: only the knobs differ (`rama_tourist_env.py`)

The tourist subclasses the art lover, so the state, action space, policy form, and reward structure are identical. Only the character changes:

Knob	Art lover	Normal tourist
importance vector	art-historical (masterpiece-concentrated)	name-recognition (famous rooms only)
dwel_per_importance	3.0 (long, ~30 min)	1.4 (short)
art_crowd_alpha	2.0 (crowd-averse)	1.0 (less averse)
decline_penalty	200	50
noshow_penalty	100	80

4.5 The matched baseline (`environment/planned_route_env.py`)

The fair “today’s museum” reference: `Discrete(2)` (stay / move-on), an 8-dimensional state (drain, importance, density, at-target, fraction-of-plan-done, time, egress, distance-to-next), and the *core* reward with *no* interventions and no booking. Same visitor character, trained and evaluated per crowd, so the booking agent’s gain is measured against an equally-optimised walk, not a straw man.

One-line summary

There are two reward sources, `q_learning.py:419` (toy) and `uffizi_env.py:833` (everything deep); the booking agents add the three terms in `rama_art_lover_env.py:355`. There are two algorithm families, value-based (Q-learning $\rightarrow$ Double-Q $\rightarrow$ DQN) and policy-based (PPO $\rightarrow$ MaskablePPO), and masking is the thread that runs through both, the masked max in the toy and the masked softmax in MaskablePPO.